

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 16

STORAGE AND FILE STRUCTURE

STORAGE AND FILE STRUCTURE: OUTLINE

Computer Storage Media

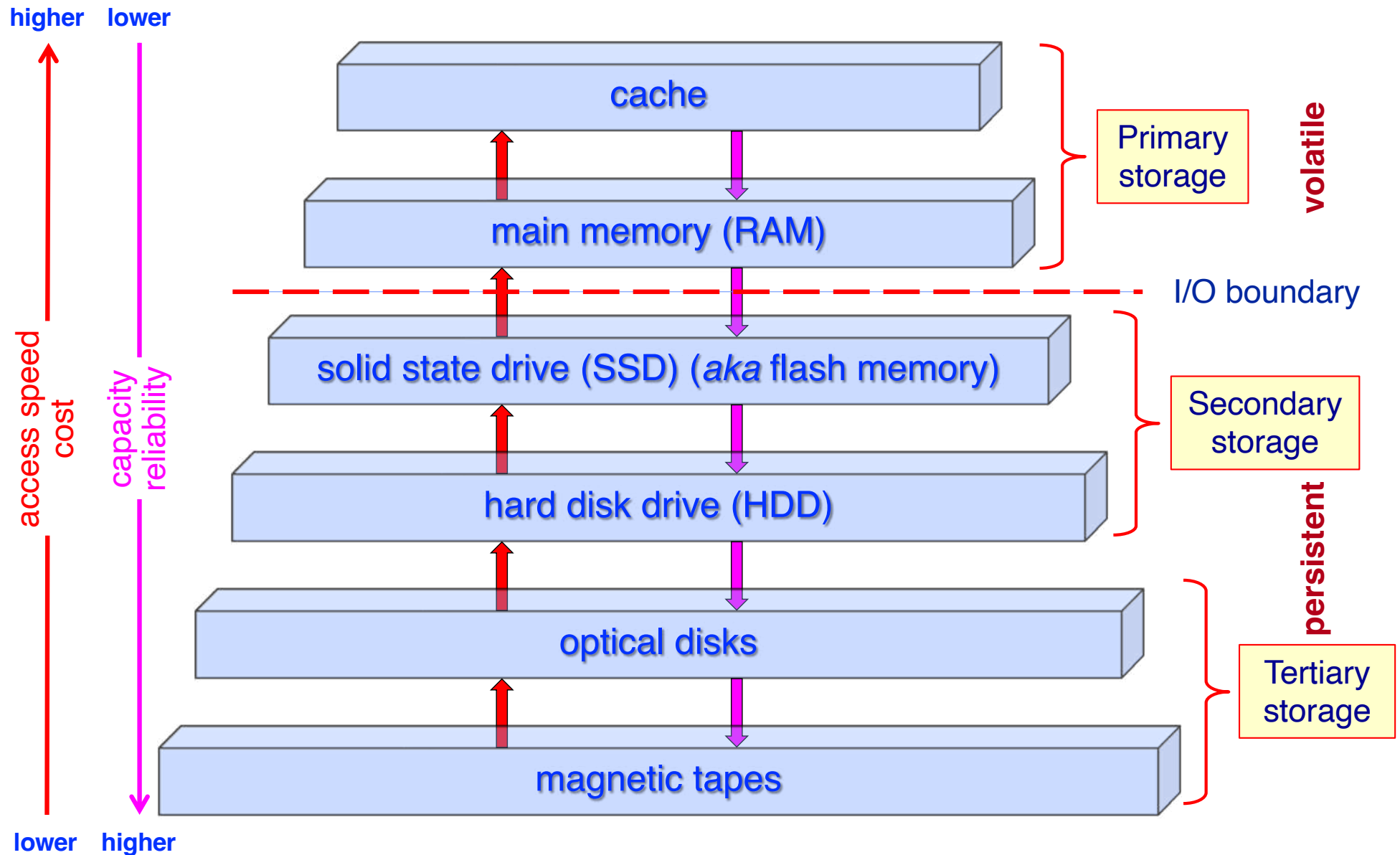
Database Buffer

Record Organization

File Organization

NOSQL DBMSs

STORAGE DEVICE HIERARCHY



STORAGE DEVICE HIERARCHY

- A typical storage hierarchy is:
 - **main memory (RAM)** for currently used data.
 - **Solid state drives (SSD aka flash memory)/hard disk drives (HDD)** for the **main database** (secondary storage).
 - **optical disks/tapes** for **archiving older versions of the data** (tertiary storage).

Why not store everything in main memory?

 Capacity **versus** Cost

 **Main memory is volatile!**
(We want the database to be saved **permanently!**)

SECONDARY STORAGE ACCESS

- Data is stored and retrieved from secondary storage in units called *pages* or *blocks* (typically 512 to 16K bytes).
- Operations on secondary storage:
 - READ:** transfer data (*input page*) from secondary storage to main memory.
 - WRITE:** transfer data (*output page*) from main memory to secondary storage.
 - Both are much slower than main memory operations, so must be planned carefully!
- The time to retrieve a page from secondary storage varies depending the type of storage device.

 **The time to access a page from an SSD is typically 2-5 times faster than from an HDD.**

DATABASE BUFFER

Data must be in main memory for an application to operate on it!

- To reduce retrieval time, DBMSs try to **minimize page transfers** between main memory and secondary storage.
- The number of accesses to secondary storage can be reduced by **keeping** as many **pages** as possible **in main memory**.

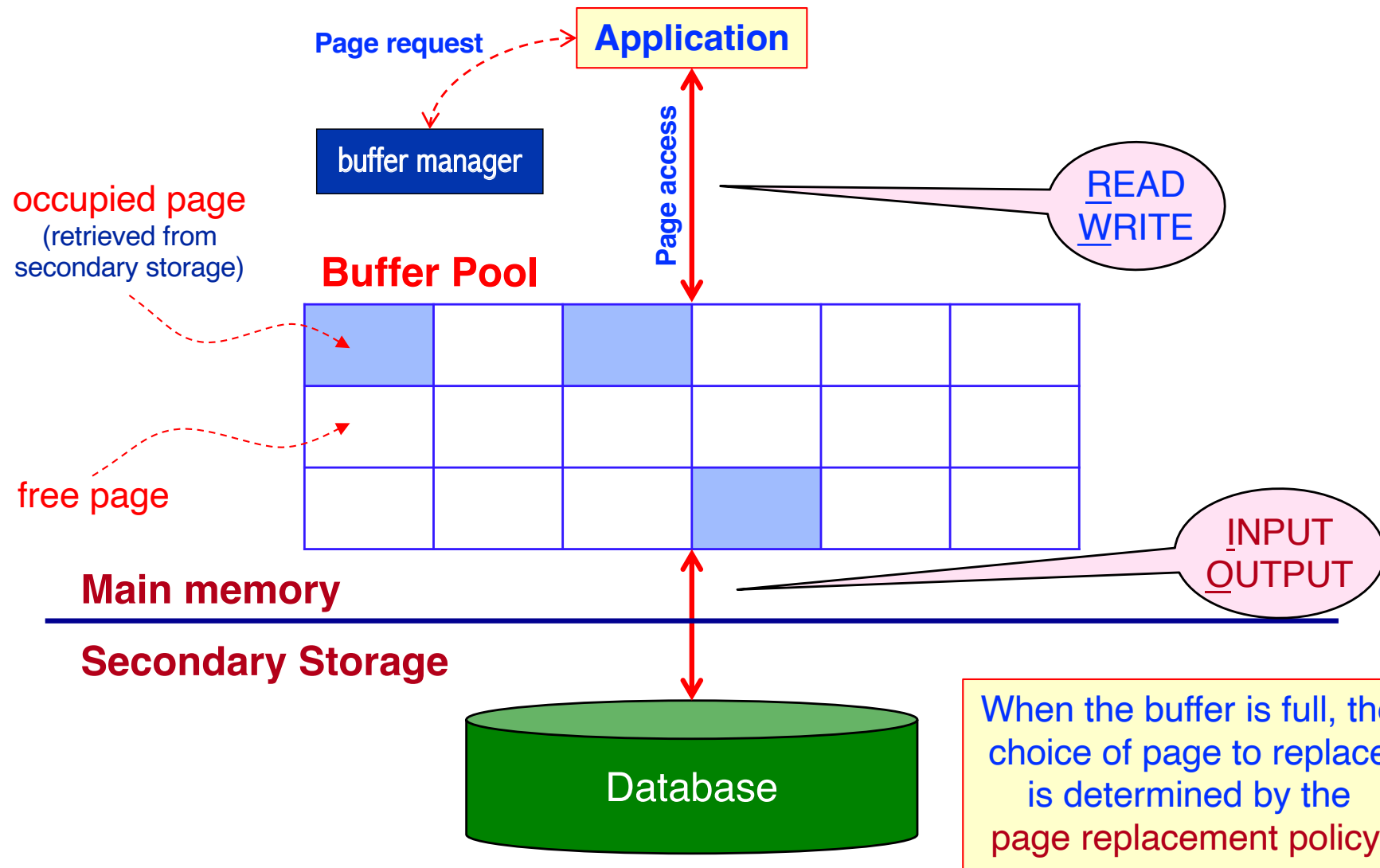
Buffer – the portion of main memory available to store **copies of secondary storage pages**.

 **NOTE: main memory space for storing pages is limited!**

Buffer manager – the operating system (OS) subsystem that manages the main memory buffer space.

 **The goal of the buffer manager is to minimize accesses to secondary storage.**

BUFFER MANAGEMENT



BUFFER MANAGER

- An application makes a **read** request to the operating system buffer manager when a page is needed from secondary storage.
- **If the page is already in the buffer**, the requesting application is **given the page's address in main memory** and accesses it directly.
- **If the page is not in the buffer:**
 - The buffer manager allocates space in the buffer for the page, replacing some other page, if necessary, to make space for the new page.
 - The page that is replaced is **written back** to secondary storage **only if** it was modified since the most recent time that it was written to/read from secondary storage.
 - Once space is allocated in the buffer, the buffer manager reads the page from secondary storage into the buffer and passes the page's address in main memory to the requesting application.
- We refer to each **page read** as an **input page access** and each **page write** as an **output page access** (or **page I/O**).

BUFFER PAGE REPLACEMENT POLICIES

- Most operating systems replace the page *least recently used* (LRU strategy).
- LRU can be a **bad strategy** for certain access patterns involving **repeated scans of data**.

Example: compute the join of two relations r and s using a nested loop.

Best in this case: replace the page *most recently used* (MRU strategy).

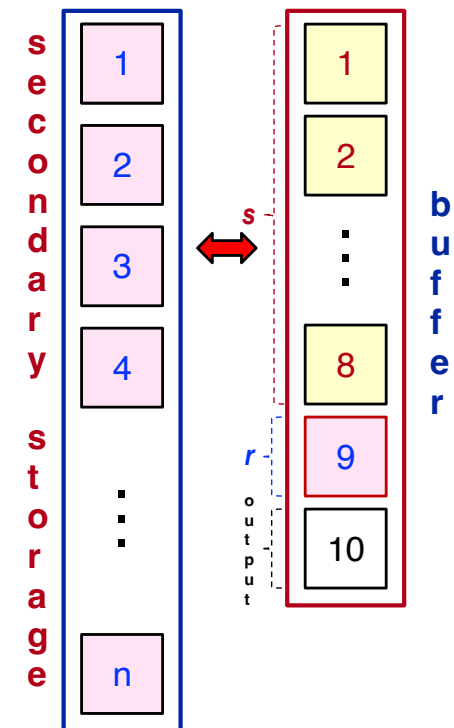
- A DBMS usually has its **own buffer manager** that **uses statistical information** regarding the probability that a request will reference a particular page to retrieve and replace pages.

Suppose the buffer can hold 10 pages and relation s needs 8 pages.

For each page in r
For each page in s

⋮

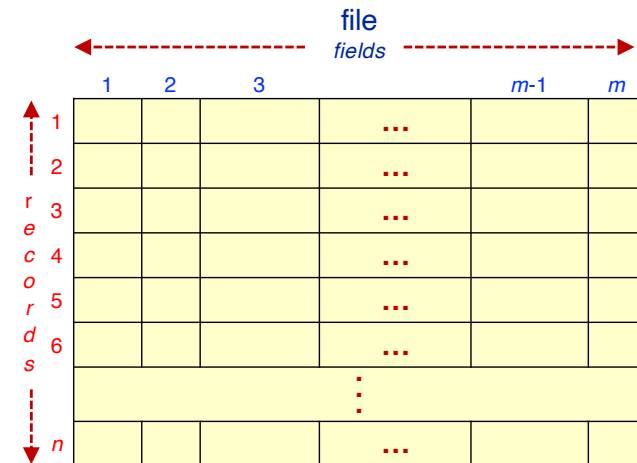
relation $r \bowtie$ relation s



RECORD ORGANIZATION

Record organization organizes data items, which usually represent attributes/fields, into physically stored records.

- A database is stored as a collection of *files* where
 - each file is a sequence of *records* and
 - each record is a sequence of *fields* that occupy several *bytes*.
- A common organization:
 - assume the **record size is fixed** (i.e., each record occupies a **fixed number of bytes**).
 - each file has **records of only one type**.
 - **different files** are used for **different types of records**.



👉 **This organization is easiest to implement.**
(Particularly useful for relational DBMSs.)

FIXED-LENGTH RECORDS: RELATIVE LOCATION

- In each page, store record i starting from byte $n * (i - 1)$, where n is the size of each record in bytes.

- Record access is simple, but records may cross (span) pages.

✎ Normally file systems do not allow records to cross page boundaries (unspanned).

✎ Consequently, there may be some unused space at the end of a page.

Catalog: account#, branchName, balance

record 1	A-102	Perryridge	400
record 2	A-305	Round Hill	350
record 3	A-215	Mianus	700
record 4	A-101	Downtown	500
record 5	A-222	Redwood	700
record 6	A-201	Perryridge	900
record 7	A-217	Brighton	750
record 8	A-110	Downtown	600
record 9	A-218	Perryridge	700

- To reclaim space when record i is deleted, shift records up.
- However, moving records inside a page is not good when records are pointed to by:
 - other records (e.g., foreign keys).
 - index entries (see next lecture).

FIXED-LENGTH RECORDS: FREE LISTS

- Do not move records in a page; instead, **store the address** of the **first deleted record** in the file header (the first record).
- Use the first deleted record to **store the address** of the **second deleted record** and so on.
- Can think of these stored addresses as **pointers** since they “point” to the location of a record.
- This is a **more space-efficient organization** since we can reuse space for **normal** fields of unused records to store the pointers. (No pointers are stored in records being used.)

Catalog: account#, branchName, balance

header			
record 1	A-102	Perryridge	400
record 2			
record 3	A-215	Mianus	700
record 4	A-101	Downtown	500
record 5			
record 6	A-201	Perryridge	900
record 7			
record 8	A-110	Downtown	600
record 9	A-218	Perryridge	700

VARIABLE-LENGTH RECORDS: BYTE-STRING REPRESENTATION

- Variable-length records arise in DBMSs due to:
 - Storage of multiple types of records in a file (*if allowed*).
 - Records that allow variable lengths for one or more fields such as strings (e.g., `varchar` data type).
 - Records that allow fields to be absent or repeat (not allowed in relational DBMSs but allowed in NoSQL DBMSs).

file

1 ±	2 ±	3 ±	4 ±	5 ±	...
-----	-----	-----	-----	-----	-----

- Simple (but bad) solution: **byte-string representation**
 - Store each record one after the other as a string of bytes.
 - Attach a special **end-of-record** (**±**) symbol to the end of each record.
 - Problems with
 - **record deletion** (results in fragmentation of free space).
 - **file growth** (may require movement of records if they are ordered).

VARIABLE-LENGTH RECORDS: EMBEDDED IDENTIFICATION

- Precede fields with **metadata** (e.g., an attribute/field name).
- Similar to the approach used for storing data in some NoSQL DBMSs.
- Requires **extra storage space**, but efficient if record size is variable or some data is absent (e.g., absent fields in MongoDB).
- Similar problems as byte-string representation.

1	branchName	Perryridge	account#	A-102	balance	400	account#	A-210	balance	900	account#	A-218	balance	700	
2	branchName	Round Hill	account#	A-305	balance	350									
3	branchName	Mianus	account#	A-215	balance	700									
4	branchName	Downtown	account#	A-101	balance	500	account#	A-110	balance	600					
5	branchName	Redwood	account#	A-222	balance	700									
6	branchName	Brighton	account#	A-217	balance	750									

VARIABLE-LENGTH RECORDS: POINTER METHOD

- Useful for certain types of records with repeating fields or multi-valued fields.
- Requires two kinds of pages in a file:

Anchor page

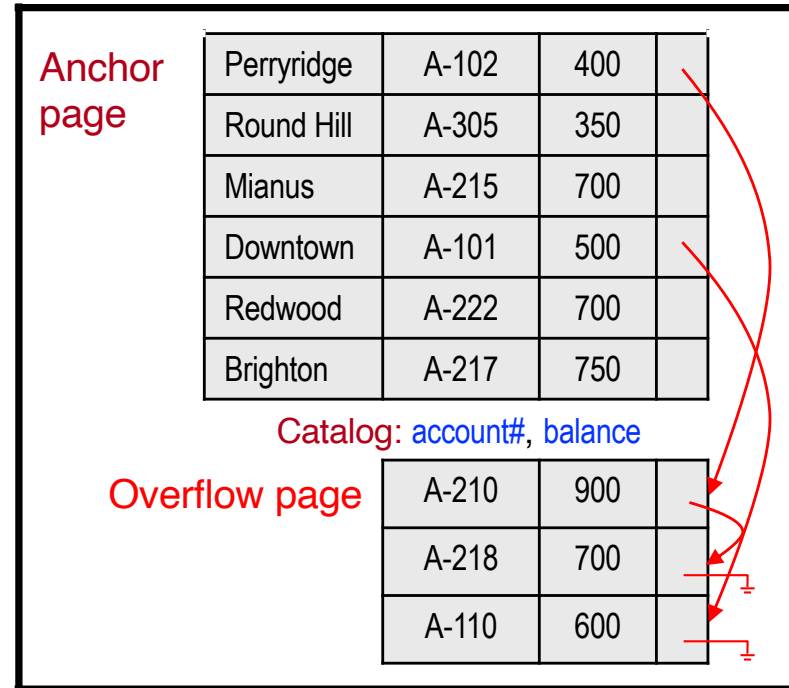
Contains the first record(s) of a chain (e.g., first account at a branch).

Overflow page

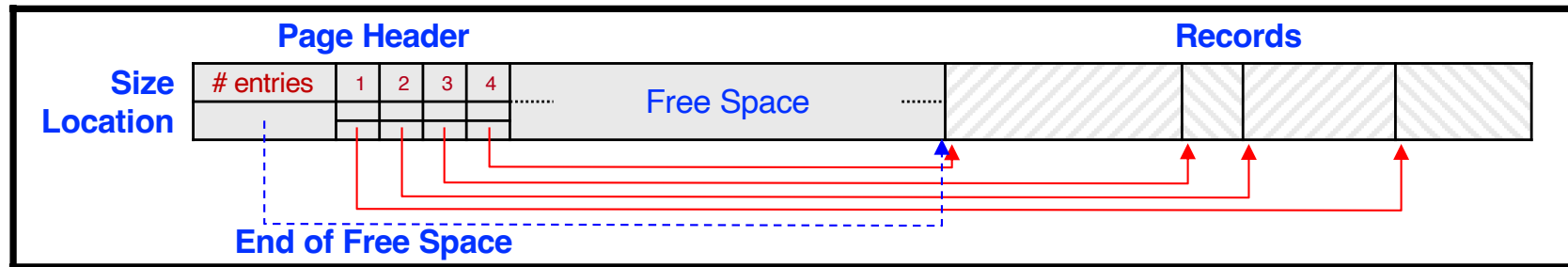
Contains records other than those that are the first records of a chain (e.g., additional accounts at a branch).

- **Pointers** link related records together.

Catalog: branchName, account#, balance



VARIABLE-LENGTH RECORDS: SLOTTED-PAGE STRUCTURE



- The **page header** contains:
 - the number of record entries.
 - the end location of the free space in the page.
 - the location and size of each record.
- Records can be moved within a page to **keep them contiguous** with **no empty space** between them.
 - ✎ **The page header must be updated when a record is moved.**
- References to records do not point directly to the records — instead, they point to the entry for the record in the page header.

FILE ORGANIZATION

Search key: one or more attributes by which records are retrieved.

- Records in a physical file are usually organized to facilitate efficient retrieval on a search key.
- If the search key is a:
 - primary or candidate key
 $\Rightarrow$ at most one record is retrieved
 - non-key
 $\Rightarrow$ multiple records may be retrieved.
- Most common file organizations:
 - heap
 - sequential
 - hash

File Blocking Factor

The blocking factor of a file r , denoted as bf_r , is the number of records that fit in a page and is equal to (for unspanned records)

$$\left\lfloor \frac{\text{\# bytes per page}}{\text{page size}} / \frac{\text{\# bytes per record}}{\text{record size}} \right\rfloor$$

Consequently, the number of pages needed to store the records in a file is equal to

$$\lceil \text{\# records} / bf_r \rceil$$

HEAP FILE ORGANIZATION

- A record can be placed anywhere in the file where there is space (usually at the end) and normally is not moved.

 **There is no relationship between a search key and a record's location.**

- To support record level operations, the DBMS needs to keep track of the:
 - pages in a file.
 - free space in pages.
 - records in a page.
- Some ways to manage this information:
 - linked list: a header page points to full and free data pages, which link to each other.
 - page directory: header pages, which link to each other, point to data pages.

HEAP FILE ORGANIZATION: OPERATIONS

- **Insertion** – add the record to the end of the file.
- **Deletion** – mark the record as deleted (i.e., deletion byte).
- **Search** – requires a **file scan** of all records regardless of the search key.
 - Except can stop the scan if the search key is a primary or a candidate key, the search is equality match, and the record is found.
- Free space in a file (i.e., deleted records) needs to be reclaimed periodically (file compaction).

SEQUENTIAL FILE ORGANIZATION

Catalog: account#, branchName, balance

A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	

Ordered by
branchName.

- The records are **stored in sequential order**, based on the **value of a search key** (usually, but not always, the primary key).

☞ Retrieval of records based on the search key is **efficient**.

☞ Retrieval based on other fields requires a **complete file scan**.

SEQUENTIAL FILE ORGANIZATION: OPERATIONS

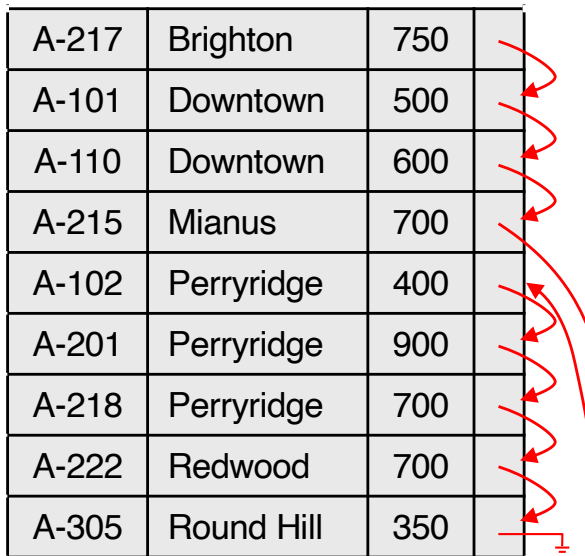
- **Insertion** – locate the position where the record is to be inserted.
 - If there is free space, insert there.
 - If no free space, insert the record in an overflow page (can be a heap file).
 - In either case, the pointer chain must be updated.
- **Deletion** – update the pointer chain.
- **Search** – do a **binary search** using the search key; else **complete file scan**.
- The file needs to be **reorganized periodically** to maintain the benefits of the sequential order.

Catalog: account#, branchName, balance

A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	

Overflow page

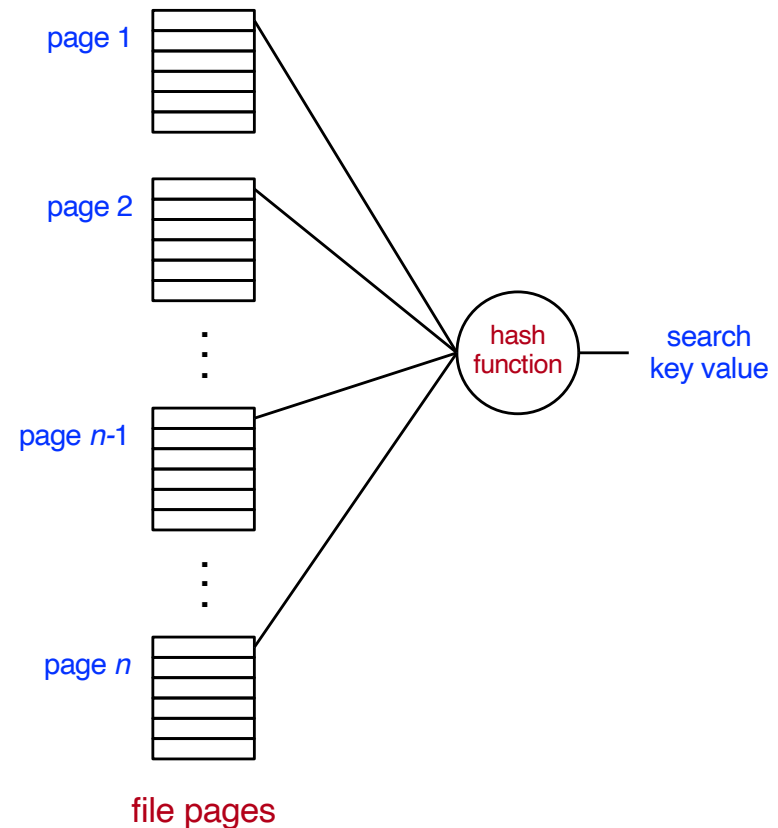
A-88	North Town	800	
------	------------	-----	--



Ordered by branchName.

HASH FILE ORGANIZATION

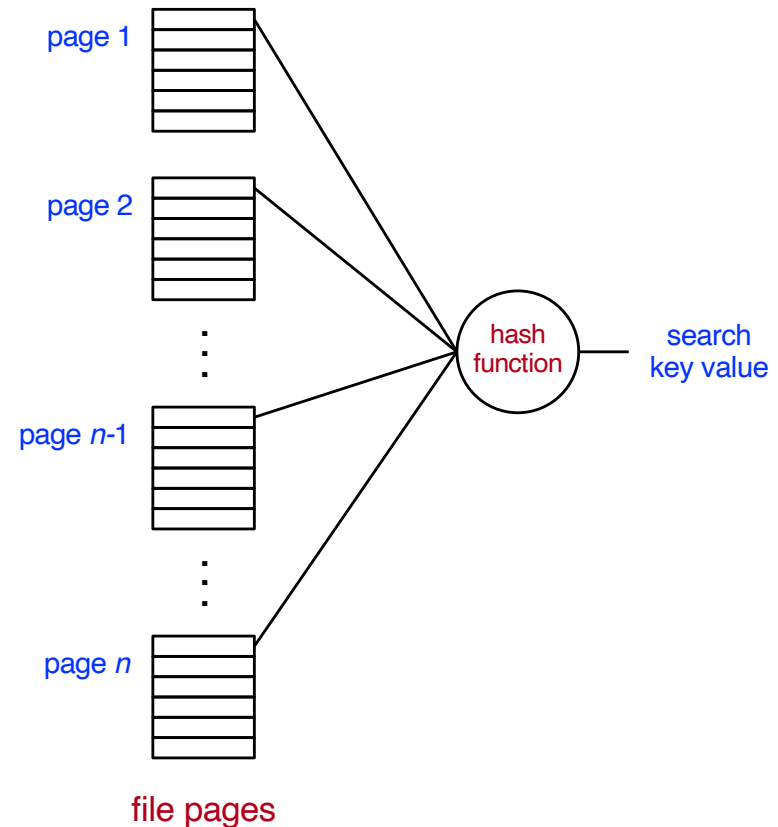
- A **hash function** defines a **value-to-(page)** address transformation such that a record's **physical page address** can be **calculated** from its **search key value**.
- The result of a hash function **determines** in which **page of the file** a **record should be stored**. (Pages are also referred to as **buckets**.)



👉 There is a **direct relationship** between the **search key value** and a record's **physical location**.

HASH FILE ORGANIZATION: OPERATIONS

- **Insertion** – apply the hash function to the search key value and store the record in the page calculated by the hash function.
- **Search/Deletion** – apply the same hash function to the search key value and retrieve/delete the record from the page calculated by the hash function.
 - A record can often be retrieved/deleted by **accessing only one page**.



EXAMPLE HASH FILE ORGANIZATION

- Assume we want to store an **Employee** relation with **100,000** records and we can put **100** records per page (i.e., the blocking factor, $bf_{Employee}$, is **100**).
- Consequently, we need $\lceil \# \text{ records} / bf_{Employee} \rceil = \lceil 100,000 / 100 \rceil = 1,000$ pages to store the records.

👉 To allow for future insertions, we could allocate **1,250 pages** for the relation (i.e., so that pages are only 80% full).

- We want to organize the file so that we can efficiently answer equality selections on salary $\Rightarrow$ **salary is the search key**.

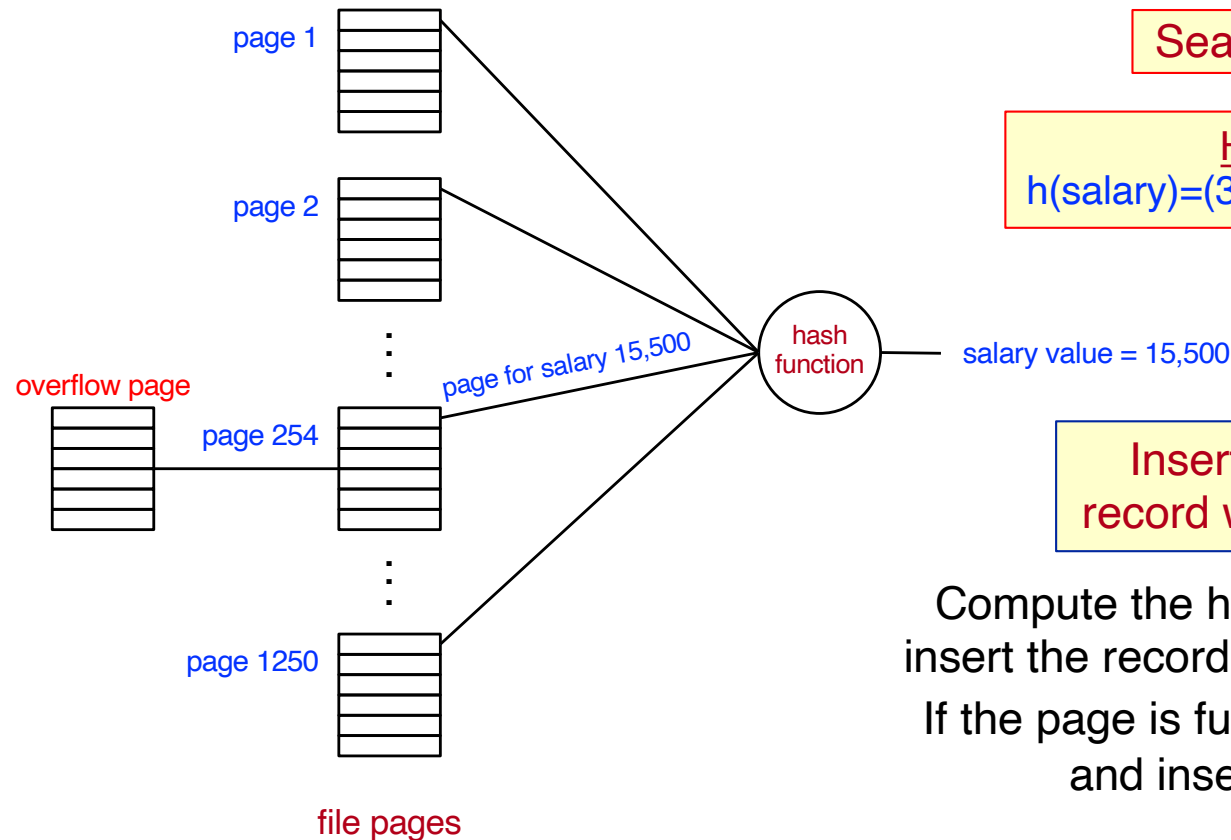
Example query: Find all employee records whose salary is equal to \$15,500.

- To support this query, the hash function might have the form

$$h(\text{salary}) = (a * \text{salary} + b) \text{ modulo } 1250.$$

(a and b are selected so that the hashed values distribute *uniformly* over the available file pages.)

EXAMPLE HASH FILE ORGANIZATION (CONT'D)



Search key: salary

Hash function
 $h(\text{salary}) = (3 * \text{salary} + 4) \text{ modulo } 1250$

Insert a new Employee
record with salary 15,500.

Compute the hash function of salary and
insert the record in the corresponding page.
If the page is full, create an *overflow page*
and insert the record there.

Query: Find all employee records
whose salary is equal to 15,500.

👉 Good for equality search
on the hash value only!

Compute the hash function of the salary and
find the records in the corresponding page.

EXAMPLE HASH FILE ORGANIZATION (CONTD)

- This organization can efficiently answer queries of the form:
Find all employees whose salary is equal to X.
- To answer this query, we compute the hash value of X and then search only in the corresponding page computed by the hash function.
- A page may contain records of employees with different salaries that produce the same hash value (e.g., employees with salary 20,500 also hash to page 254). This is not a problem because, since we retrieve the page into the buffer, we can check all its records in main memory and select only the ones that satisfy the query condition.
- If there are **no overflow pages**, **answering the query requires that just one page be read** (i.e., **one page I/O**).
-  **If there are overflow pages, we must retrieve all of them. Why?**
- Hashing is not good for range search (e.g., **Find all employees with salaries between 15,000 and 16,000**) because the records of these employees may be stored in many different pages.

NOSQL DBMSS

- NoSQL DBMSs are distributed systems that employ hundreds to thousands of servers (nodes).
 - To achieve **good performance**, in the presence of high workloads, **records are partitioned** across multiple servers.
 - To support **high availability**, in the presence of server failures, **records are distributed** (and **replicated**) on several servers.

DATA STORAGE: HASHING

- For **key-value** based NoSQL DBMSs, the **key** is hashed to determine where to store the record in a file.

Example: Map sailor id (the key) and sailor record (the value).

Key (sailorId)	Value
22	{sName: 'Dustin', hkid: 'A298456(6)', rating: 7, age: 45}
32	{sName: 'Andy', hkid: 'M028391(4)', rating: 8, age: 25}
64	{sName: 'Horatio', hkid: 'F710943(2)', rating: 7, age: 35}
71	{sName: 'Zorba', hkid: 'P910562(4)', rating: 1, age: 16}



hash function

hash table

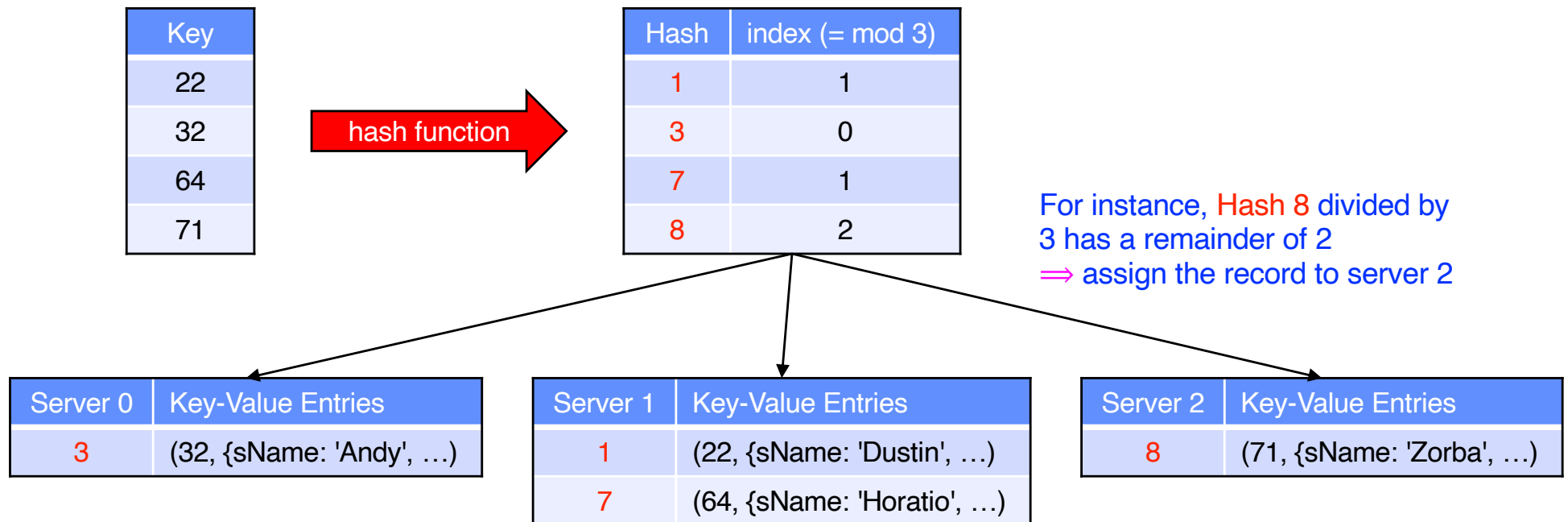
Hash value	Key-Value Entries
1	(22, {sName: 'Dustin', hkid: 'A298456(6)', rating: 7, age: 45})
3	(32, {sName: 'Andy', hkid: 'M028391(4)', rating: 8, age: 25})
7	(64, {sName: 'Horatio', hkid: 'F710943(2)', rating: 7, age: 35})
8	(71, {sName: 'Zorba', hkid: 'P910562(4)', rating: 1, age: 16})

DATA DISTRIBUTION: SHARDING

Sharding: the process of partitioning a large database into smaller pieces and assigning the smaller pieces to servers.

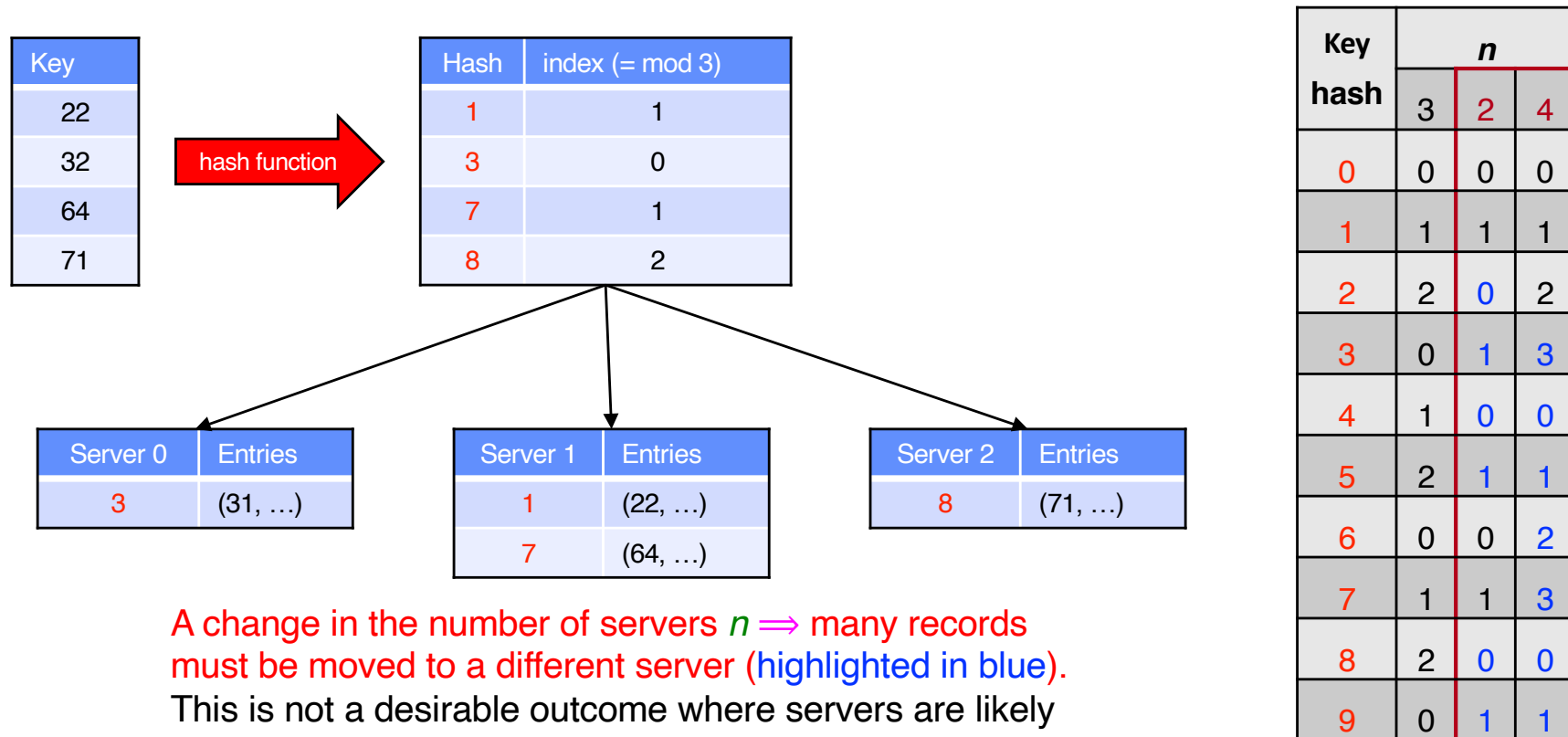
Example: Hash every key hash value to a **server identifier**.

$\text{server index}(\text{hash}) = \text{mod}(\text{hash value}, \text{number of servers})$
(i.e., a hash of key hash values)



DATA DISTRIBUTION: SHARDING (CONT'D)

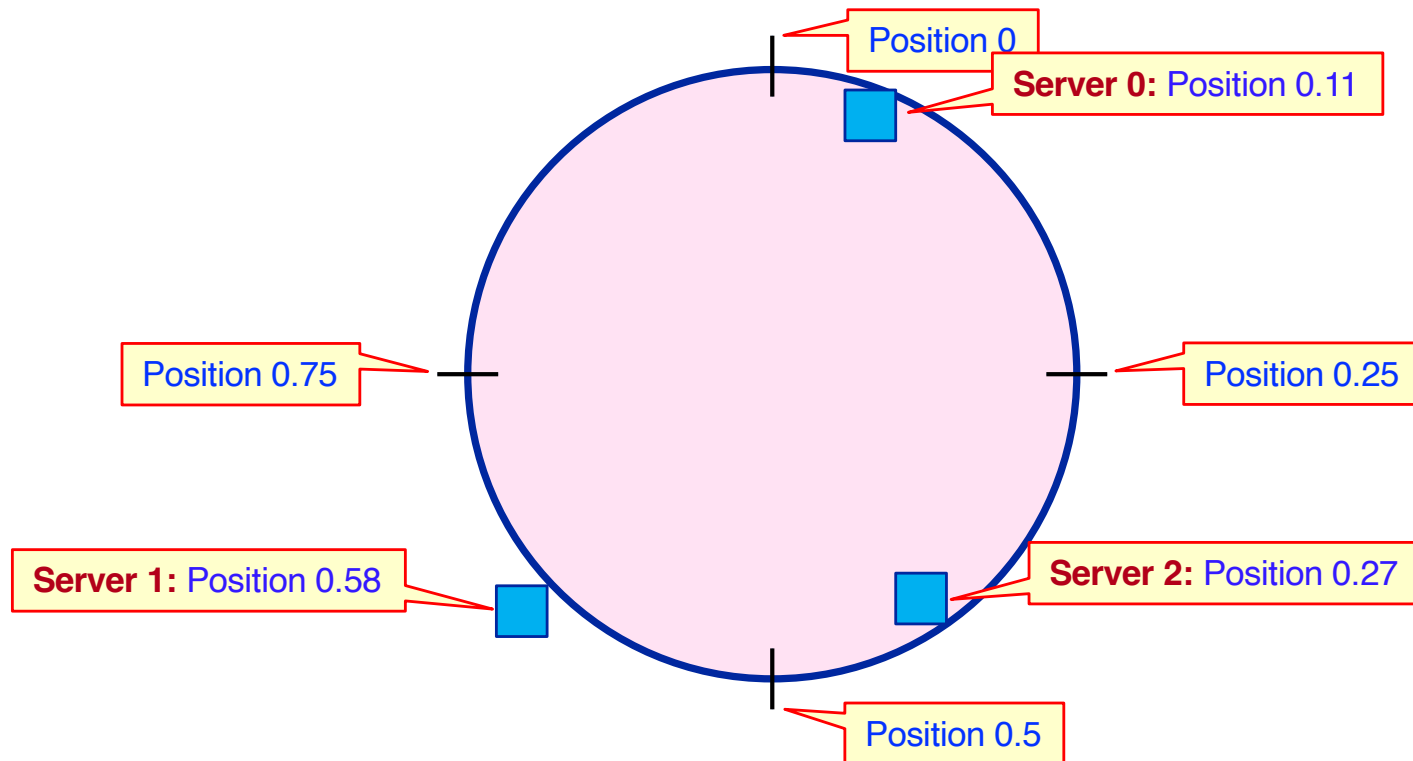
Problem: need to remap data to servers as the number of servers change.



Solution: consistent hashing

DATA DISTRIBUTION: CONSISTENT HASHING

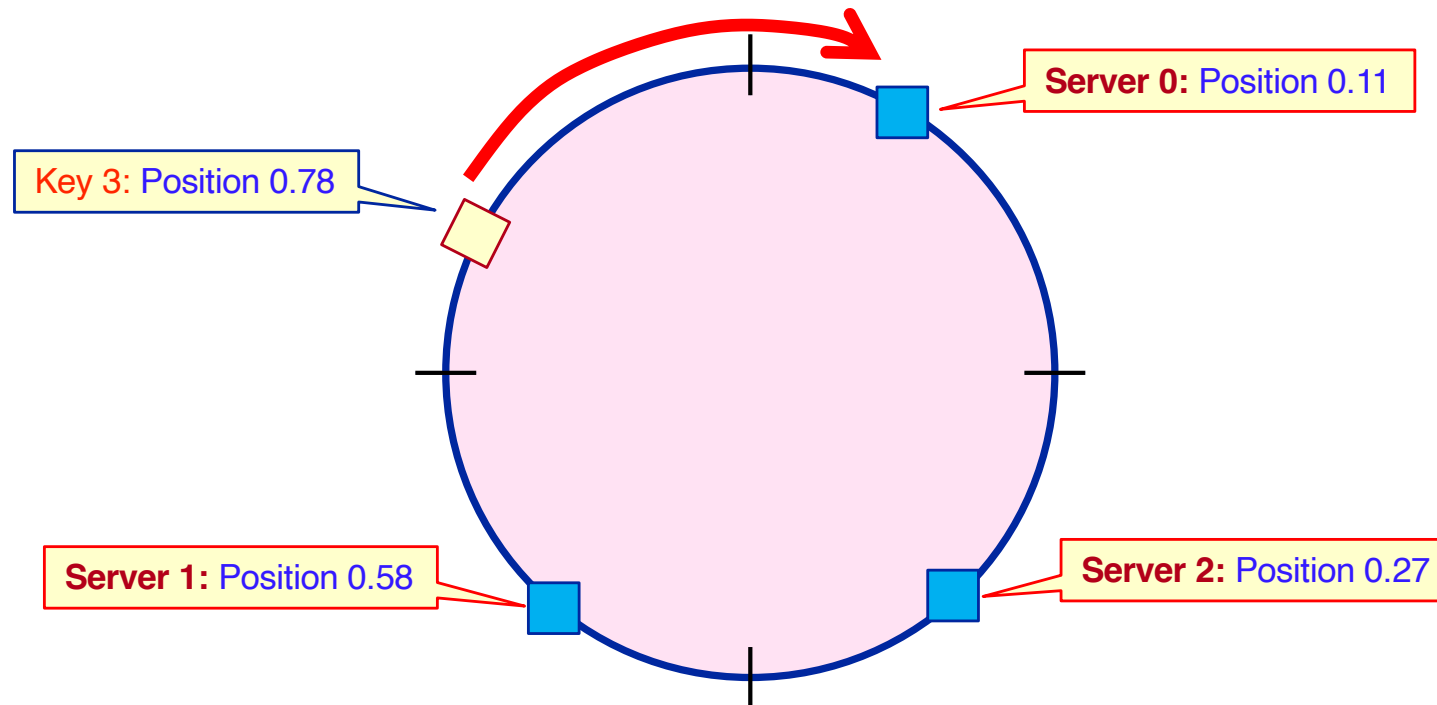
- At the core of a **consistent hashing** scheme is a so-called (conceptual) **ring-topology**, which is basically a representation of the number range $[0,1]$ as shown in the figure.



- All three servers with identifiers **0**, **1**, **2** are hashed to place them in **a position on this ring**.

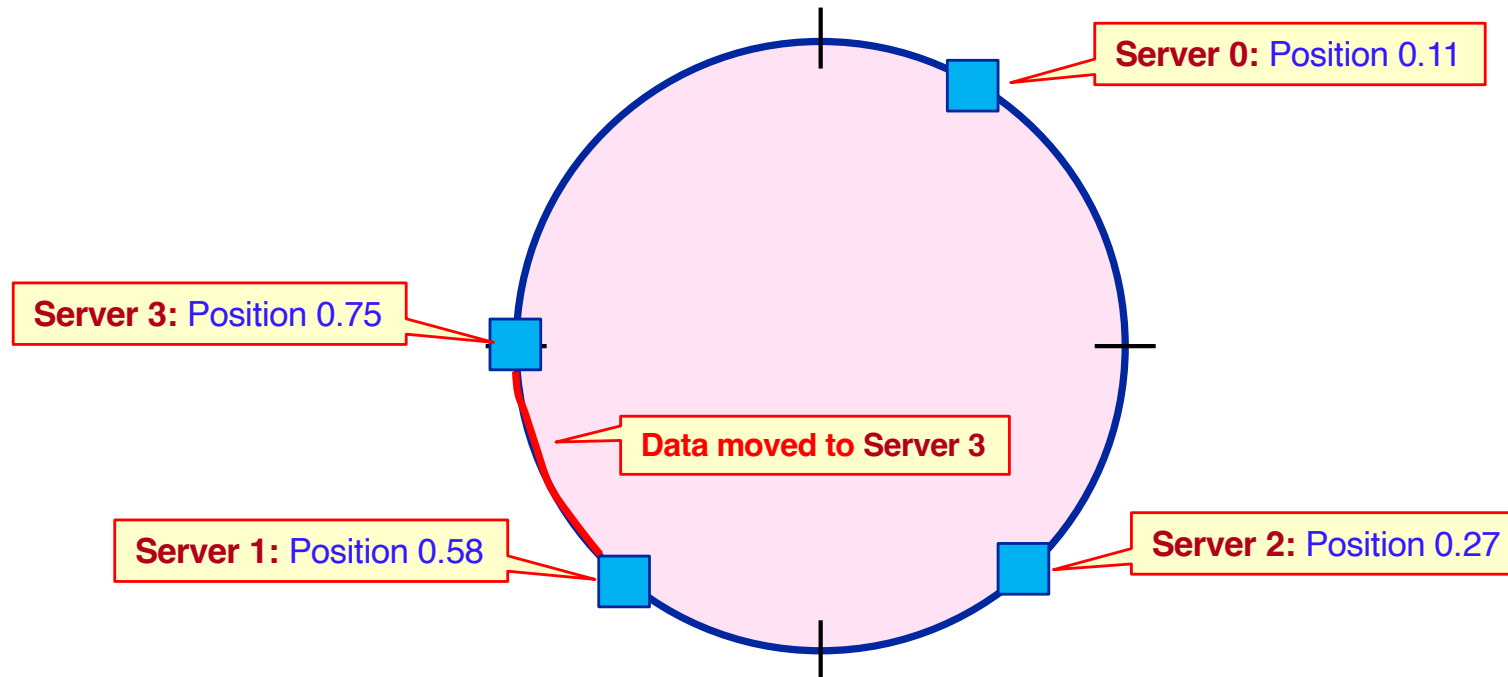
DATA DISTRIBUTION: CONSISTENT HASHING (CONTD)

- Then, each key is hashed to a position on the ring and the actual **key-value** pair is stored on the **first server that appears clockwise** from the hashed point on the ring.



DATA DISTRIBUTION: CONSISTENT HASHING (CONTD)

- If a new server, **Server 3**, is added to the ring, only the keys positioned on the red highlighted section of the ring need to be moved to the new server.



✎ Typically, only need to move about $k/(n+1)$ keys where k is the number of keys and n the number of servers.

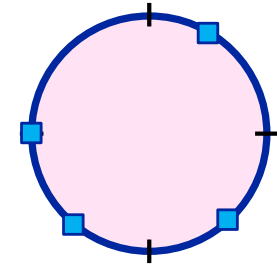
DATA DISTRIBUTION: CONSISTENT HASHING (CONT'D)

- Due to the uniformity property of a “good” hash function, roughly $1/n$ key-value pairs will end up being stored on each server.
- Most of the key-value pairs will remain unaffected if a server is added or removed.
- Typically, the fraction of keys that need to be moved when using this scheme is about $k/(n+1)$ where k is the number of keys and n the number of servers.
- This is a much smaller fraction than is the case for modulo-based hashing.

DATA DISTRIBUTION: REPLICATION

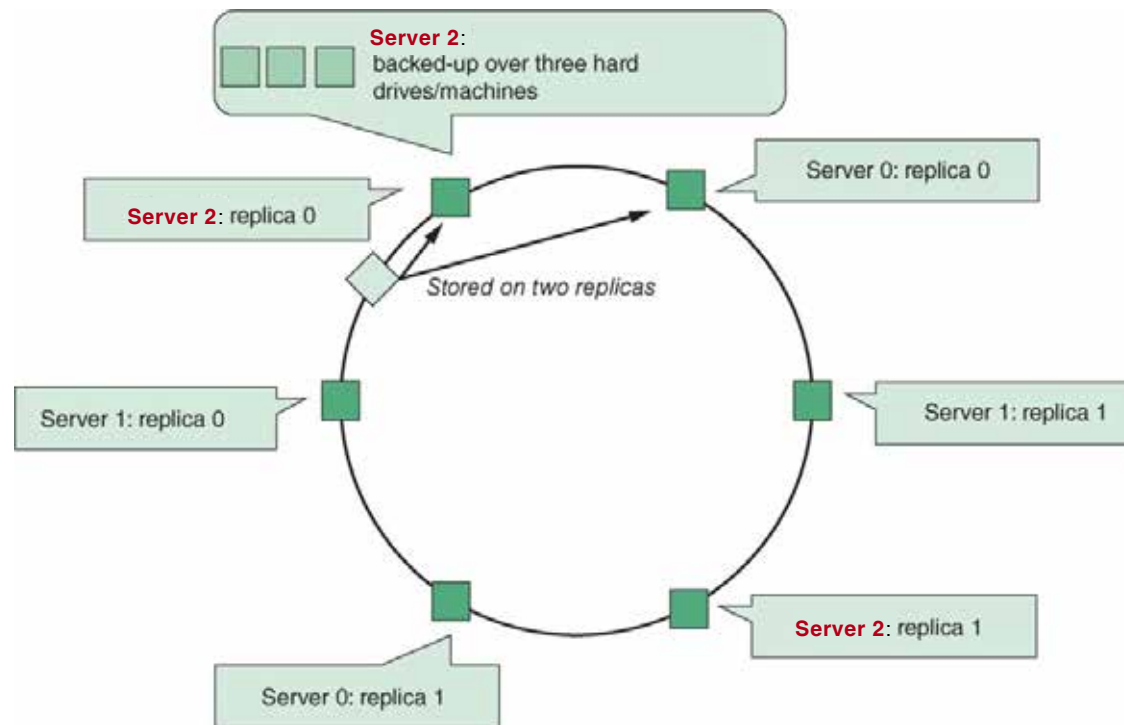
- Problems with consistent hashing:
 - If two servers are mapped too close to one another, one of these ends up with few keys to store.
 - When a server is added, all the keys moved to this new server originate from just one other server.
- Instead of mapping a server S to a single point on the ring, it is mapped to multiple positions, called (logical) replicas.
 - For each physical server S , we end up with r points (the number of replicas) on the ring, called virtual nodes.

Note: Each replica (virtual node) still represents the same physical instance.
- The consistent hashing mechanism can be extended to replicate key-value pairs across multiple servers.
 - E.g., we can store a key-value pair on two or more servers clockwise from the key's position on the ring.



DATA DISTRIBUTION: REDUNDANCY

- As well as creating (logical) replicas, a full redundancy scheme can be set up in which each server itself corresponds to **multiple physical servers**, each storing a fully redundant copy of the data.
 - E.g., **Server 2** can be backed up (full redundant copy) over three hard drives or servers.



SIMPLISTIC ANALYSIS OF FILE ORGANIZATIONS

- When analyzing and comparing different file organizations, for simplicity we will:
 - ignore CPU costs (e.g., the cost of searching a page in memory).
 - approximate page I/O costs by ignoring I/Os saved due to page prefetching.
 - usually do average-case analysis using several simplistic assumptions.
 - Single record insert and delete
 - For heap files
 - Equality selection is on the primary key; therefore, there is exactly one match.
 - A record insert is always at the end of a file.
 - For sequential files
 - File compaction happens after a record deletion.
 - Selection is on sort field(s).
 - For hash files
 - No overflow pages.
 - 80% page occupancy.

PAGE I/O COST OF FILE OPERATIONS

file	1	2	3	4	5	6	... pages ...	B
------	---	---	---	---	---	---	---------------	---

Operation	Heap File	Sequential File	Hash File
Scan all records	B	B	B^1
Equality search ²	0.5 B	$\log_2 B^3$	1
Range search	B	$\log_2 B +$ # of pages with matches	B^1
Insert	2^4	Equality search + B^5	2
Delete	Equality search + 1	Equality search + B^5	2

B is the number of pages in a file.

¹ Assumes 100% page occupancy. For 80% page occupancy, 1.25B pages are needed to store all records.

² Assumes the search is on the key value.

³ Assumes binary search is used.

⁴ Assumes the record is inserted at the end of the file – read last page and write it back.

⁵ Assumes insert/delete is in the middle of the file and, therefore, need to read and write all pages in second half of the file.

 **Several assumptions underlie these (rough) estimates!**

STORAGE AND FILE STRUCTURE: SUMMARY

- Available **data storage** options have **different cost/performance**.
 - ✎ For large databases, **HDDs** are still the most used **DBMS storage device**.
- **DBMS performance** may be **highly dependent on** the **retrieval** of records **from secondary storage**.
 - ✎ Often the bottleneck in **DBMS performance**.
 - ✎ **Buffer management** is very important.
- **DBMSs** use file organizations provided by operating systems to store data.
 - ✎ **heap, sequential or hash**.